Passo a Passo do empilhamento de memória (recursividade)

```c
#include <stdio.h>
// Função que imprime os dígitos de um número na ordem inversa
void inverterNumero(int n) {
    // Caso base: se o número for 0, não faz nada e começa a voltar
    if (n == 0) {
        return;
    }
    // Ação: imprime o último dígito (resto da divisão por 10)
    printf("%d", n % 10);
    // Chamada recursiva: passa o número sem o último dígito
    inverterNumero(n / 10);
}
int main() {
    int valor = 12345;
    printf("Invertendo o numero %d: ", valor);
    inverterNumero(valor);
    printf("\n");
    return 0;
}
```

1° Passo: O programa inicia na função main, a variável “valor” é criada e recebe o valor “12345”, sendo alocado na memória 24

4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
								valor						
								12345						

40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74	78

7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0	B4

B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0

2° Passo: A função inverterNumero é chamada pela primeira vez. A variável “n” recebe o valor “12345”. O programa imprime o dígito 5.

4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
			n					*valor*						
			12345					12345						

40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74	78

7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0	B4

B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0

Passo 3: Nova chamada recursiva. A variável “n” recebe o valor “1234”. O programa imprime o dígito 4.

4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
			n					*valor*						
			12345					12345						

40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74	78
			n											
			1234											

7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0	B4

B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0

Passo 4: Nova chamada recursiva. A variável “n” recebe o valor “123”. O programa imprime o dígito 3.

4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
			n					*valor*						
			12345					12345						

40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74	78
			n											
			1234											

7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0	B4
			n											
			123											

B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0

Passo 5: Nova chamada recursiva. A variável “n” recebe o valor “12”. O programa imprime o dígito 2.

4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
			n					*valor*						
			12345					12345						

40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74	78
			n		*n*									
			1234		12									

7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0	B4
			n											
			123											

B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0

Passo 6: Nova chamada recursiva. A variável “n” recebe o valor “1”. O programa imprime o dígito 1.

4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
			n					*valor*						
			12345					12345						

40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74	78
			n		*n*									
			1234		12									

7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0	B4
			n		*n*									
			123		1									

B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0

Passo 7: Nova chamada recursiva. A variável “n” recebe “0”. A condição inicial é atingida e a função começa a retornar.

4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
			n					*valor*						
			12345					12345						

40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74	78
			n		*n*									
			1234		12									

7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0	B4
			n		*n*									
			123		1									

B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0
					n									
					0									

Passo 8: A função inverterNumero(0) é encerrada e seu espaço na pilha é liberado.

4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
			n					*valor*						
			12345					12345						

40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74	78
			n		*n*									
			1234		12									

7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0	B4
			n		*n*									
			123		1									

B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0

Passo 9: A execução retorna para n = 1. A função é finalizada e o espaço de memória é liberado.

4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
			n					*valor*						
			12345					12345						

40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74	78
			n		*n*									
			1234		12									

7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0	B4
			n											
			123											

B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0

Passo 10: Retorno da chamada para n = 12. O espaço correspondente é removido da pilha.

4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
			n					*valor*						
			12345					12345						

40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74	78
			n											
			1234											

7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0	B4
			n											
			123											

B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0

Passo 11: Retorno da chamada para n = 123. O espaço de memória é liberado.

4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
			n					*valor*						
			12345					12345						

40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74	78
			n											
			1234											

7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0	B4

B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0

Passo 12: Retorno da chamada para n = 1234. O espaço de memória é liberado.

4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
			n					*valor*						
			12345					12345						

40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74	78

7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0	B4

B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0

Passo 13: Retorno da chamada inicial da função inverterNumero. O último espaço na pilha é liberado.

4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
								valor						
								12345						

40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74	78

7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0	B4

B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0

Passo 14: O controle retorna para a função main. O programa finaliza sua execução.

4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
								valor						
								12345						

40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74	78

7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0	B4

B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0